NXT WAVE

WELCOME

CCBP 4.0

21st Century Skills

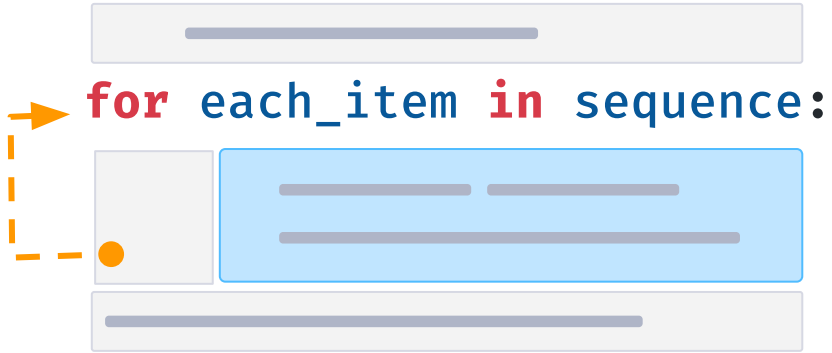
String Methods

Recap

For Loop

for statement iterates over each item of a sequence

- Sequence of Characters (string)
- Sequence of numbers, etc.



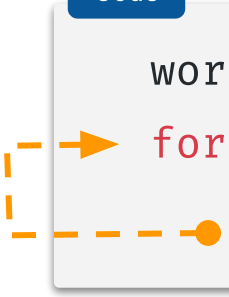
Recap

For Loop

```
for each_item in sequence:
    Repeating
    Block of Code
```

Code

```
word = "Python"
for each_char in word:
    print(each_char)
```



Recap

Range

Generates a sequence of numbers starting from this



```
range(start, end)
```



Stops before **end**

(**end** is not included)

Recap

Sequence of Integers

```
range(5, 10)
```



Gives sequence of integers

Recap

Iterate Over Numbers

Code

```
for number in range(5, 8):
    print(number)
```

Output

```
5
6
7
```

Daily Challenge

Identify The Mistake

Code

```
a = input()
len_of_a = len(a)
b = ""
for i in range(1,len_of_a):
    b = b + "-" + a[i]
print(b)
```

Input

```
Python
```

Output

```
P-y-t-h-o-n
```

Daily Challenge

Identify The Mistake

Code

```
a = input()
len_of_a = len(a)
b = a[0]
for i in range(1,len_of_a):
    b = b + "-" + a[i]
print(b)
```

Input

```
Python
```

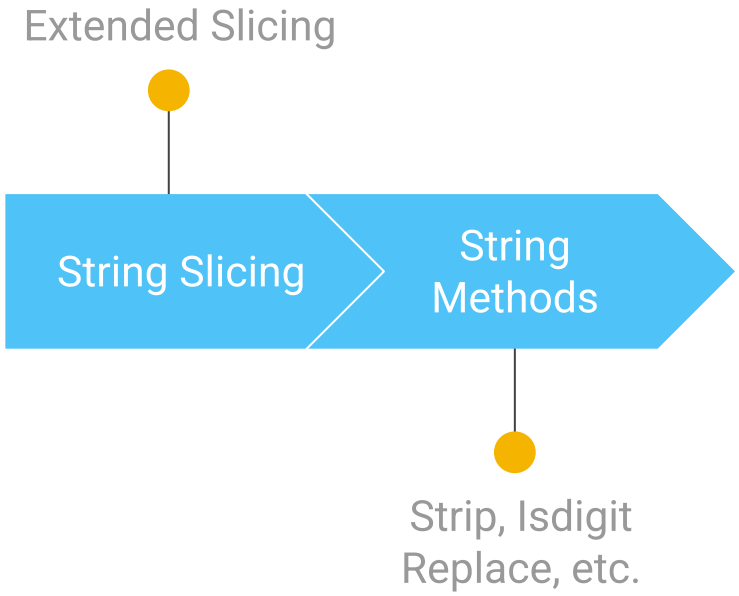
Output

```
P-y-t-h-o-n
```

NXT WAVE

Agenda for Today's Session

String Methods



String Slicing

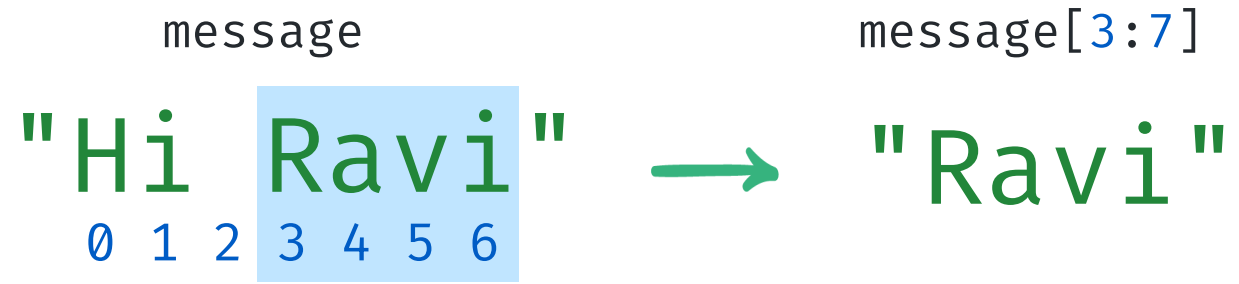
Obtaining a part of a string is called

String Slicing



Obtain Part of a String

Slicing

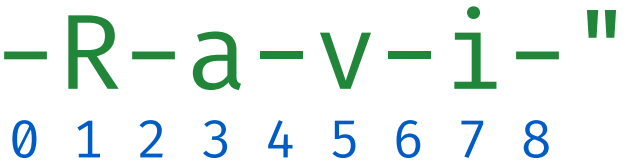


String Slicing

Accessing Parts of a String

Can we extract the name `"Ravi"` from this string?

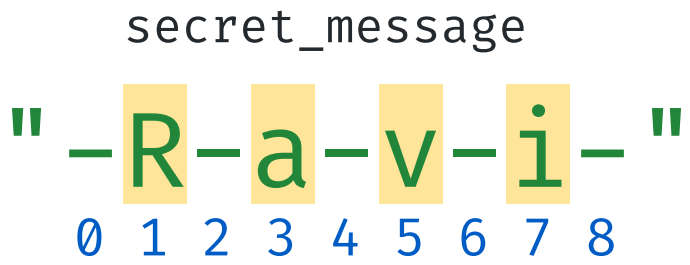
`secret_message`



String Slicing

Accessing Parts of a String

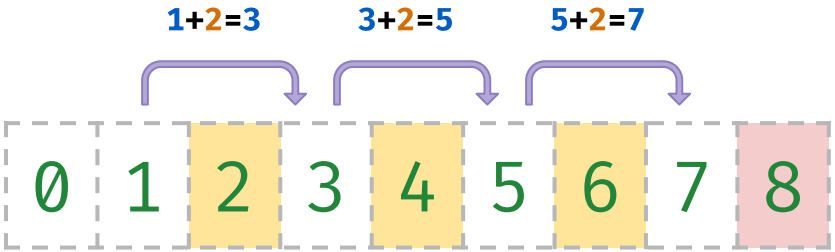
To extract name from this string, we need alternate characters starting from index 1



Extended Slicing

```
variable[start_index:end_index:step]
```

Step determines the increment between each index for slicing.

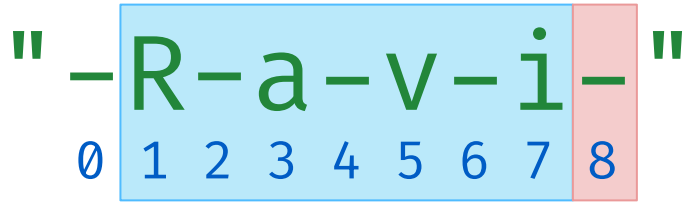



```
[1:8:2]
```

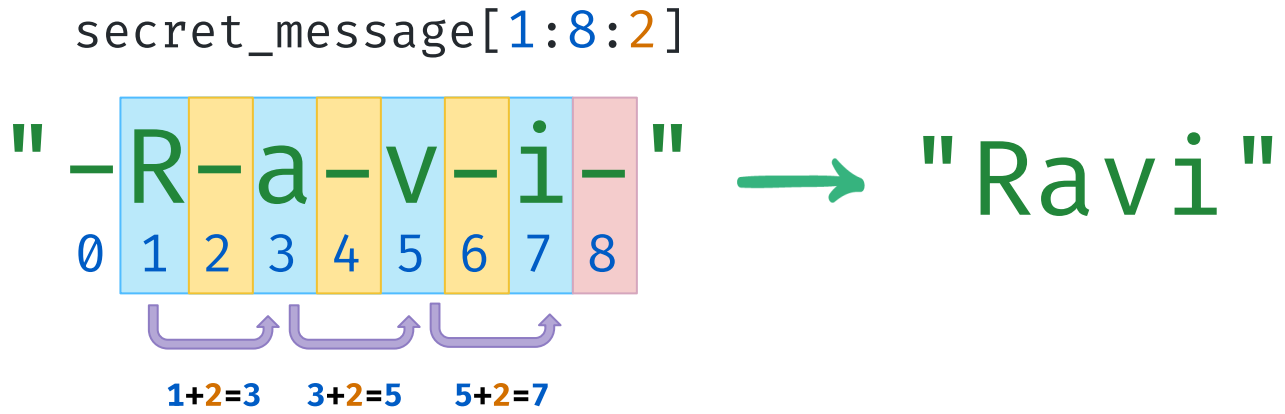
Strings

Extended Slicing

```
secret_message[1:8:2]
```



Extended Slicing



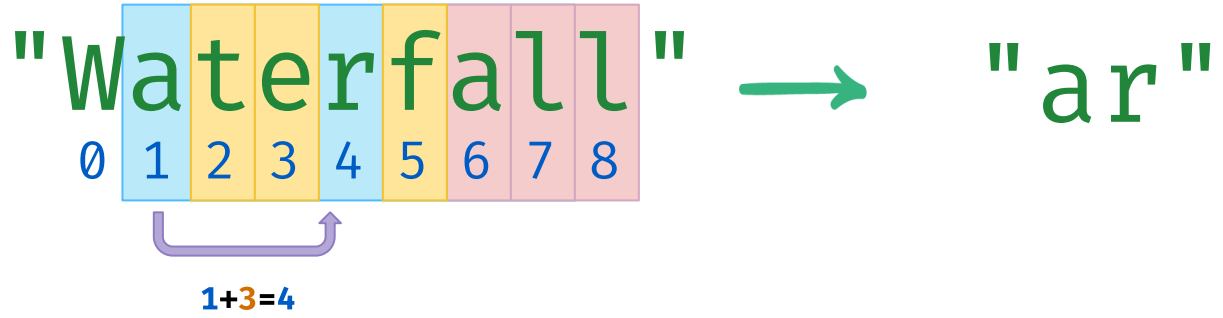
Strings

Extended Slicing

Code

```
a = "Waterfall"
```

a[1:6:3]



Strings

Slicing

Code

```
a = "Waterfall"
part = a[1:6:3]
print(part)
```

Output

```
ar
```

String Methods

Introduction to String Methods

Methods

Python has a set of

built-in reusable utilities

They simplify the most commonly

performed operations

String Methods

```
.isdigit()
.lower()
.startswith()
.replace()
```

```
.strip()
.upper()
.endswith()
```

and more...

Scenario

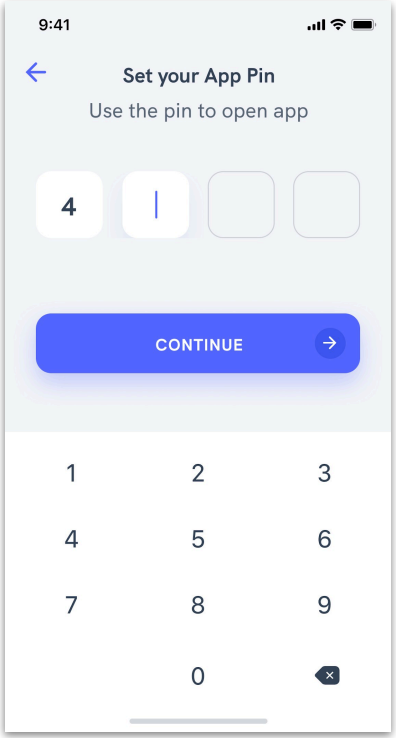
String Methods

Let's say,

you want to set a 4-digit PIN

We need to check if

the PIN entered by a user is valid

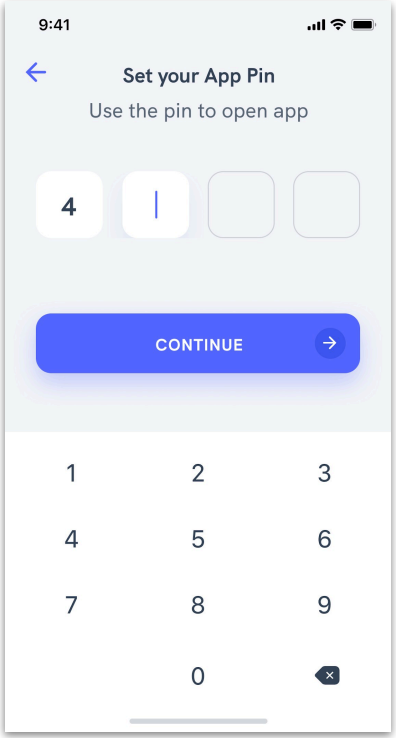


Scenario

String Methods

Processing

- Check if each character of the input is a number
- Check if the length of the input is 4 digits



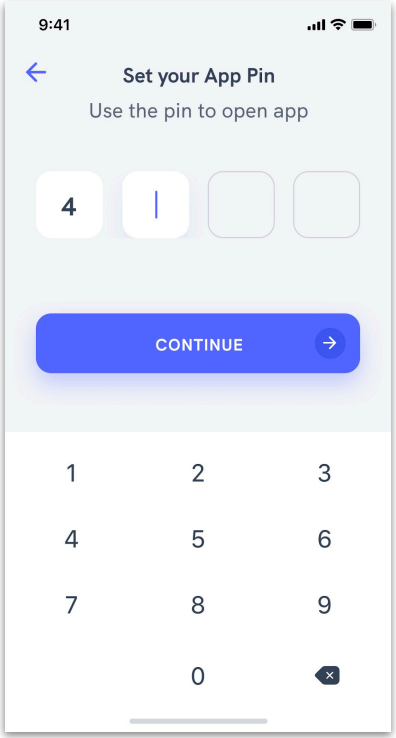
Scenario

String Methods

Processing

- Check if each character of the input is a number
- Check if the length of the input is 4 digits

String method **isdigit** simplifies this



String Methods

Isdigit

```
str_var.isdigit()
```

Gives **True** if
all the characters are digits.
Otherwise, **False**

Code

```
pin = "4748"
is_digit = pin.isdigit()
is_4_digit = (len(pin) == 4)
is_valid = is_digit and is_4_digit
print(is_valid)
```

String Methods

Isdigit

Code

```
pin = "4748"
is_digit = pin.isdigit()
is_4_digit = (len(pin) == 4)
is_valid = is_digit and is_4_digit
print(is_valid)
```

Output

```
True
```

String Methods

Isdigit

Code

```
pin = "984a"
is_digit = pin.isdigit()
is_4_digit = (len(pin) == 4)
is_valid = is_digit and is_4_digit
print(is_valid)
```

Output

```
False
```

Scenario

String Methods

The user input may contain extra spaces

```
" Ravi "
```

We have to remove these spaces from both the ends of a string

```
" 9876543210 "
```

String Methods

Strip

Code

```
mobile_1 = " 9876543210 "
mobile_2 = "9876543210"
is_same = (mobile_1 == mobile_2)
print(is_same)
```

Output

```
False
```

String Methods

Strip

```
str_var.strip()
```

Removes all the leading and trailing spaces from a string

Code

```
mobile = " 9876543210 "
mobile = mobile.strip()
print(mobile)
```

String Methods

Strip

Code

```
mobile = " 9876543210 "
mobile = mobile.strip()
print(mobile)
```

Output

```
9876543210
```

String Methods

Strip

```
str_var.strip(chars)
```

We can also specify `characters` that need to be removed

Code

```
name = "Ravi."
name = name.strip(".")
print(name)
```

String Methods

Strip

Code

```
name = "Ravi."
name = name.strip(".")
print(name)
```

Output

```
Ravi
```

String Methods

Strip

Removes specified characters that are leading or trailing

Code

```
name = "M.Ravi."
name = name.strip(".")
print(name)
```

Output

```
M.Ravi
```

String Methods

Strip

Removes all space, comma(,) and full stop(.) that lead or trail the string

Code

```
name = ", .. ,, ravi ,, .. ."
name = name.strip(" ,.")
print(name)
```

String Methods

Strip

Code

```
name = ", .. ,, ravi ,, .. ."
name = name.strip(" ,.")
print(name)
```

Output

```
ravi
```

Scenario

String Methods

Let's say there's a spelling mistake in a document

```
"teh cat and teh dog"
```

We have to replace `"teh"` with the correct spelling

String Methods

Replace

```
str_var.replace(old,new)
```

Gives a new string after replacing all the occurrences of the old substring with the new substring

Code

```
sentence = "teh cat and teh dog"
sentence = sentence.replace("teh","the")
print(sentence)
```

String Methods

Replace

Code

```
sentence = "teh cat and teh dog"
sentence = sentence.replace("teh","the")
print(sentence)
```

Output

```
the cat and the dog
```

Scenario

String Methods

Let's say we have to check if the given URL is a secure URL

`"https://onthegomodel.com"`

We have to check if the input is starting with `"https://"`

String Methods

Startswith

```
str_var.startswith(value)
```

Gives **True**

if the string starts with

the specified value.

Otherwise, **False**

Code

```
url = "https://onthegomodel.com"
is_secure_url = url.startswith("https://")
print(is_secure_url)
```

NXT WAVE

String Methods

Startswith

Code

```
url = "https://onthegomodel.com"
is_secure_url = url.startswith("https://")
print(is_secure_url)
```

Output

```
True
```

Scenario

String Methods

Let's say we have to check if the input given by user is a Gmail ID

We have to check if the input is ending with `"@gmail.com"`

```
"rahul123@gmail.com"
```

String Methods

Endswith

```
str_var.endswith(value)
```

Gives **True**

if the string ends with

the specified value.

Otherwise, **False**

Code

```
gmail_id = "rahul123@gmail.com"
is_gmail = gmail_id.endswith("@gmail.com")
print(is_gmail)
```

String Methods

Endswith

Code

```
gmail_id = "rahul123@gmail.com"
is_gmail = gmail_id.endswith("@gmail.com")
print(is_gmail)
```

Output

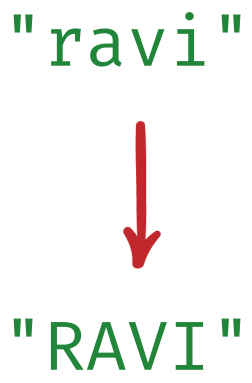
```
True
```

Scenario

String Methods

Let's say we have to read username
to fill in an application form

Whatever might be the user input,
we want to convert the string into uppercase



String Methods

Upper

```
str_var.upper()
```

Gives an uppercase string of the given string

Code

```
name = "ravi"
print(name.upper())
```

String Methods

Upper

Code

```
name = "ravi"
print(name.upper())
```

Output

```
RAVI
```

Code

```
name = "AyUsH"
print(name.upper())
```

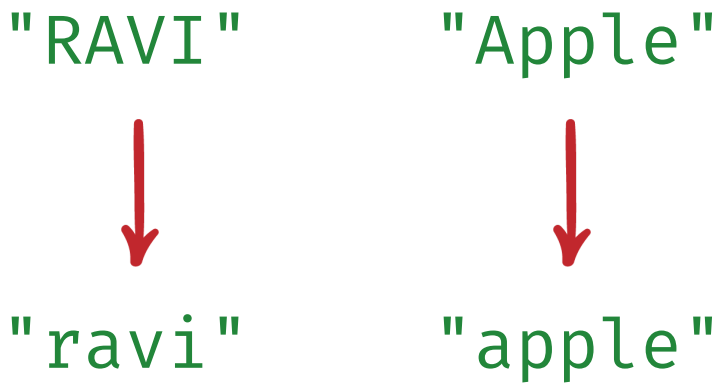
Output

```
AYUSH
```

Scenario

String Methods

Similarly, we can convert the string to lowercase using string method `lower()`



String Methods

Lower

```
str_var.lower()
```

Gives a lowercase string of the given string

Code

```
name = "RAVI"
print(name.lower())
```

String Methods

Lower

Code

```
name = "RAVI"
print(name.lower())
```

Output

```
ravi
```

Code

```
fruit = "Apple"
print(fruit.lower())
```

Output

```
apple
```

String Methods

Key Takeaways

- Extended Slicing
- String Methods
  - Strip
  - Isdigit
  - Replace



Key Takeaways

- String Methods
  - Startswith
  - Endswith
  - Lower
  - Upper

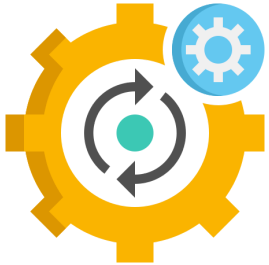


Practice



Step - 1

Checking System Requirements



Feedback



NXT WAVE

THANK YOU

NXT WAVE

ALL THE BEST